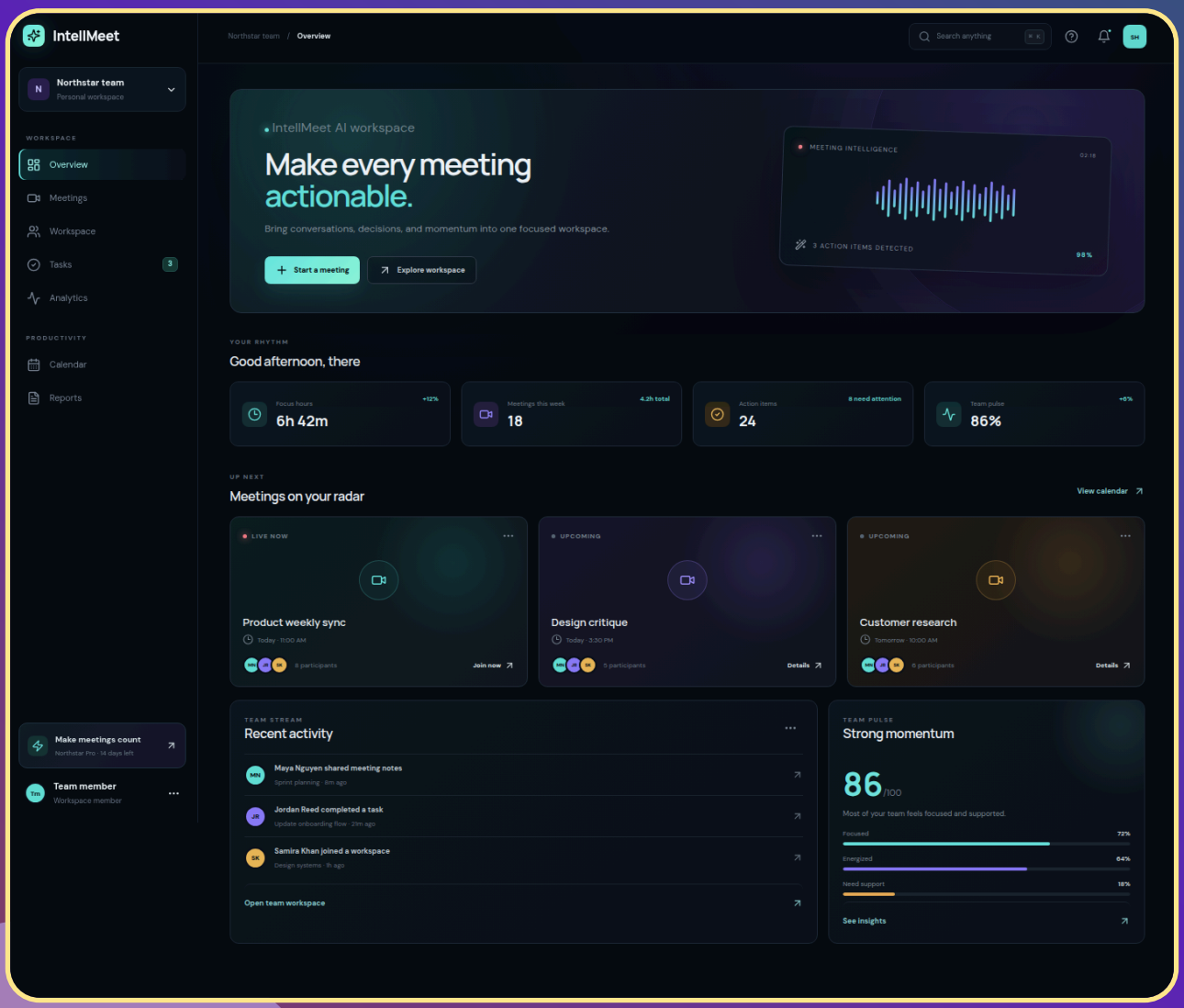
ZIDIO · WEB DEVELOPMENT PROJECT

IntellMeet

AI-Powered Enterprise Meeting & Collaboration Workspace

A project report covering the product overview, implemented user experience, authentication backend, system architecture, validation, and deployment readiness.



PREPARED BY

Shubham Singh

24 September 2026

PROJECT STATUS

Functional prototype · Copilot added

Contents

1 Executive summary 1
2 Project overview 1
2.1 Problem statement 1
2.2 Intended users and objectives 1
3 User experience and screenshots 2
3.1 Login and registration 2
3.2 Dashboard 3
3.3 Meeting discovery and room concept 5
3.4 Team workspace, tasks, and analytics 7
4 Feature behavior and persistence 9
5 Technology stack 9
6 System architecture 10
6.1 Registration and sign-in sequence 12
7 Data model and backend procedures 12
8 Security and engineering quality 12
9 Validation and deployment status 13
9.1 Checks completed 13
9.2 Deployment status 13
10 Limitations and recommended next steps 13
11 Conclusion 14
12 Alignment with the Zidio web brief 14
13 References 15

1 Executive summary

IntellMeet is a responsive meeting-workspace prototype for teams that need a clear path from scheduled conversation to follow-up work. The user interface brings together account access, a dashboard, meeting discovery, a meeting-room concept, workspace and action-item views, and illustrative analytics.

This updated build adds email registration and sign-in to the existing React and Express application. The server validates registration input, normalizes email addresses, hashes passwords with bcrypt, stores the profile and credential in MySQL or TiDB through Drizzle, and issues a signed HTTP-only session cookie. Signed-out visitors are redirected to the login route. A profile-menu action clears the session on sign-out.

> **Current delivery status.** The source type-checks, all six automated tests pass, the production build succeeds, and the MySQL migration has been generated. Live database persistence and public hosting are not verified because this package was supplied without a database URL or hosting credentials. The dashboard and meeting content remain prototype data rather than account-specific backend records.

Client	Server and API	Data and auth
React 19 · TypeScript · Vite	Express 4 · tRPC 11	Drizzle · MySQL/TiDB · bcrypt · signed sessions

2 Project overview

2.1 Problem statement

Meetings can produce decisions without clear ownership. When schedules, notes, follow-up tasks, and team communication are split across tools, participants must reconstruct context after the call. IntellMeet demonstrates a consolidated workspace in which meeting context and the next actions appear in one product flow.

2.2 Intended users and objectives

The prototype is aimed at team members who plan or join recurring discussions and team leads who need to review follow-up activity. Its objectives are to make the product purpose legible, give users a navigable meeting workflow, connect meeting concepts to tasks and team activity, and provide an authentication entry point backed by server code.

Objective	Evidence in this build	Status
Account access	Registration, login, session check, sign-out UI and server procedures	Implemented; DB setup required
Meeting workflow	Dashboard, meeting list, room drawer, workspace and tasks	Interactive prototype
Responsive use	Mobile-aware login/register layout and dashboard breakpoints	Implemented in UI

3 User experience and screenshots

3.1 Login and registration

The login page gives returning users an email/password entry point and an account-creation route. Registration collects a name, email, and password. Client-side form constraints provide quick feedback; the server repeats validation and enforces its own password and email rules. Authentication errors remain on the form instead of starting the legacy external OAuth flow.

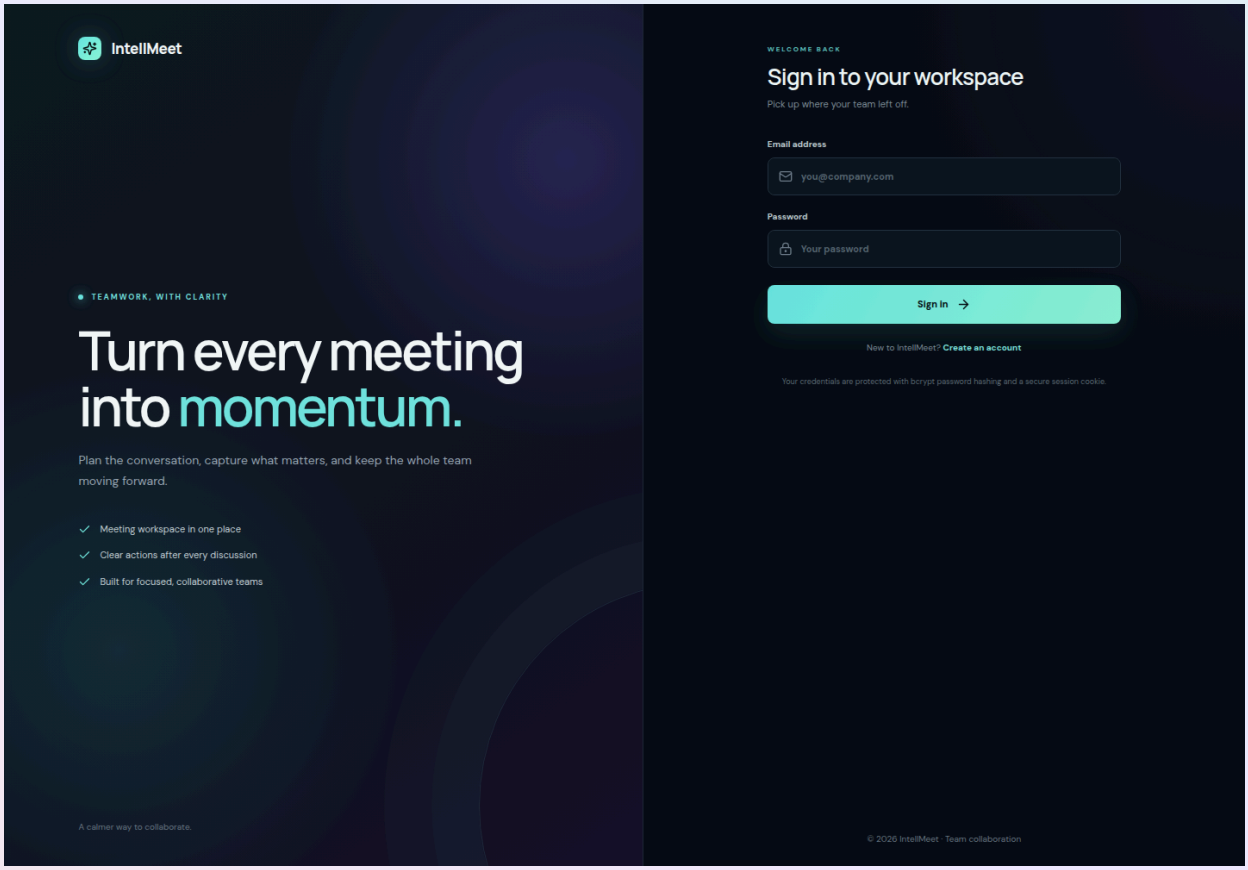


Figure 1: Login screen captured from the updated local application. The screenshot documents the interface; it does not prove a live database is configured.

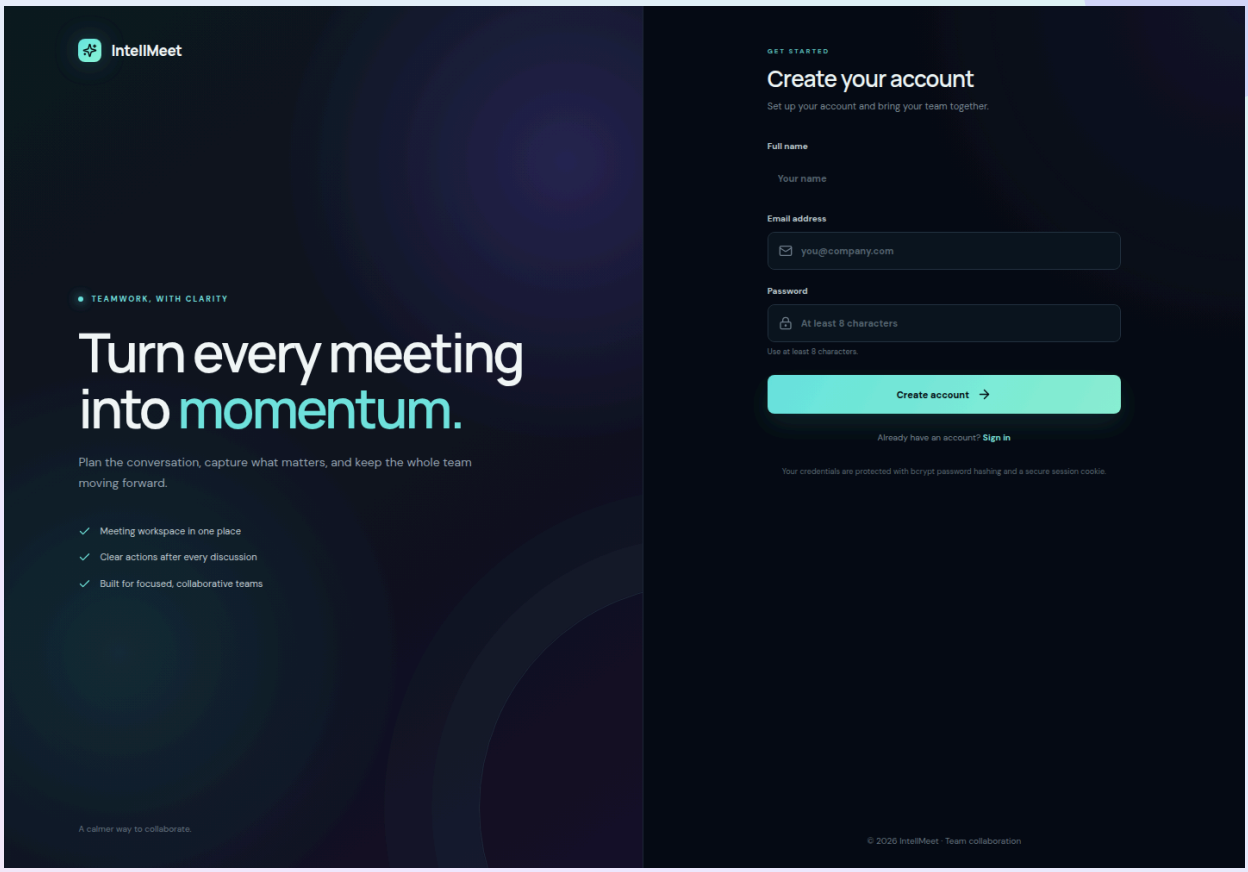


Figure 2: Registration screen with required name, email, and password fields.

3.2 Dashboard

The dashboard is the product's operational home. It presents the IntellMeet workspace identity, sample meeting and focus metrics, meeting cards, team activity, and a team-pulse panel. User identity is now drawn from the signed-in account. The statistics and activity shown in this screenshot are illustrative prototype content.

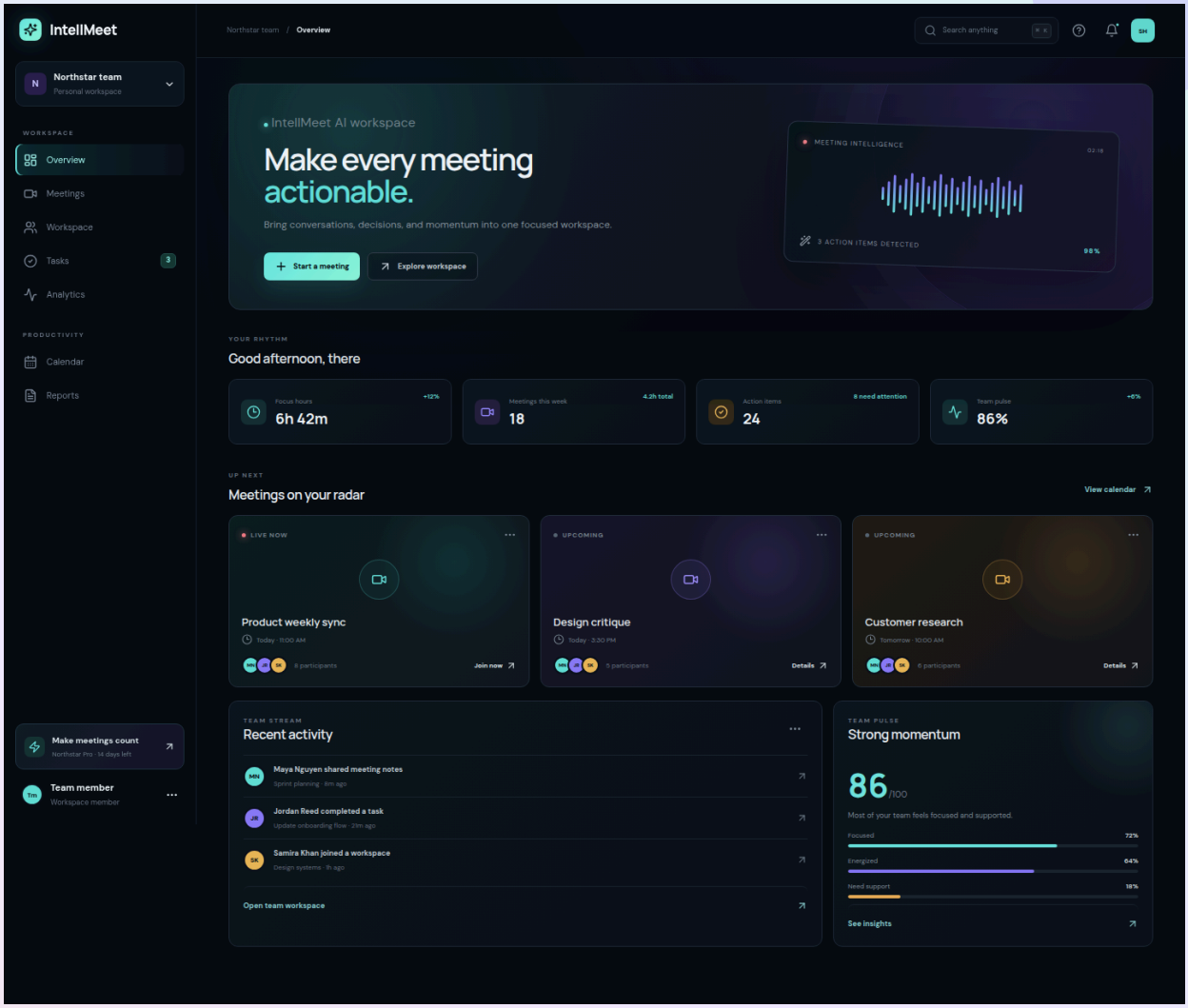


Figure 3: Current dashboard screen. Labels and values shown are sample dashboard data, not telemetry from a connected team.

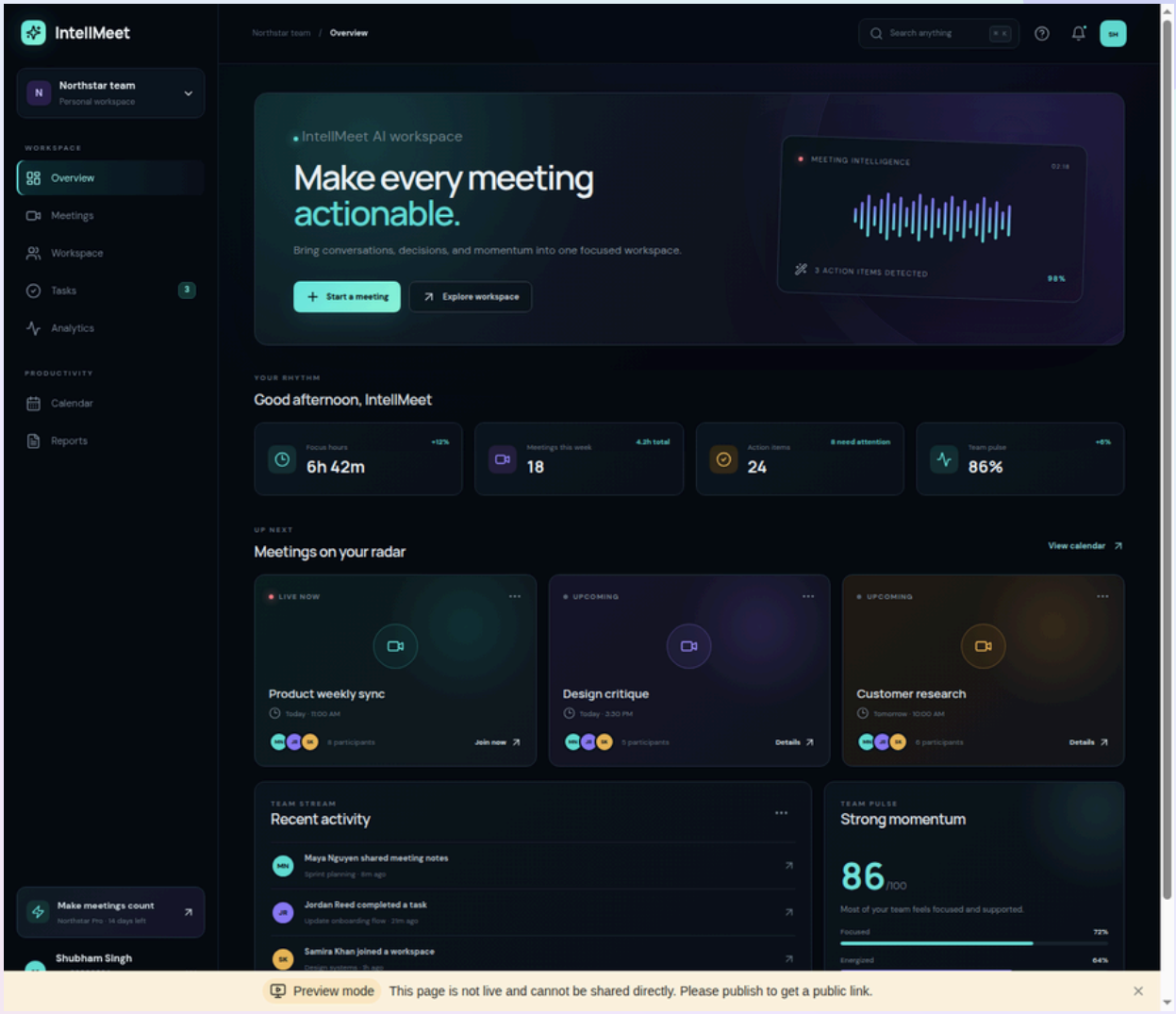


Figure 4: Verified signed-in dashboard for Shubham Singh. The workspace profile and email identity are visible after successful registration.

3.3 Meeting discovery and room concept

The Meetings view separates scheduled sessions from the overview. It provides upcoming and past filters, search, period selection, meeting cards, and a meeting-start action. The room drawer demonstrates the intended meeting layout with sample controls, transcript, copilot panel, and chat. It is not connected to live media, persistent transcripts, or an external AI service.

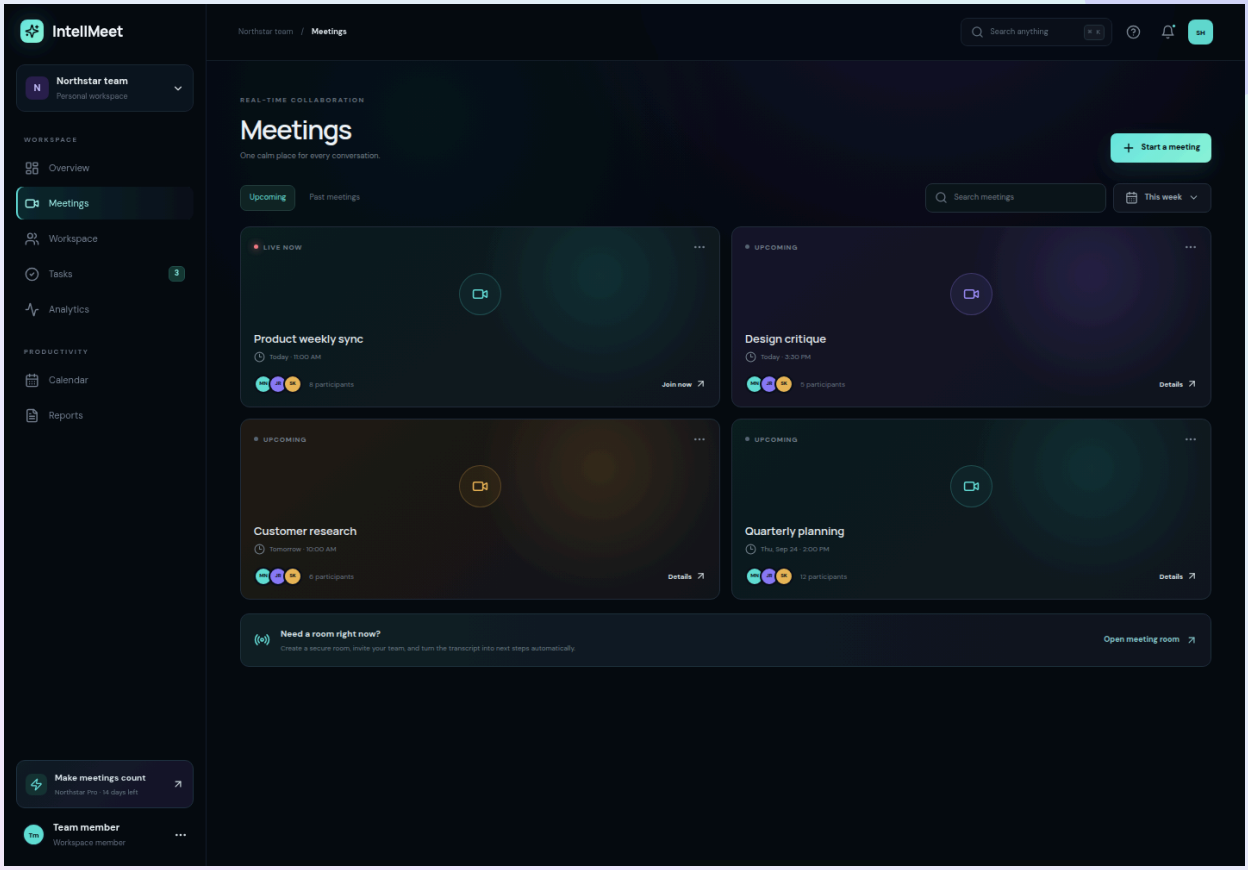


Figure 5: Meeting schedule and discovery controls.

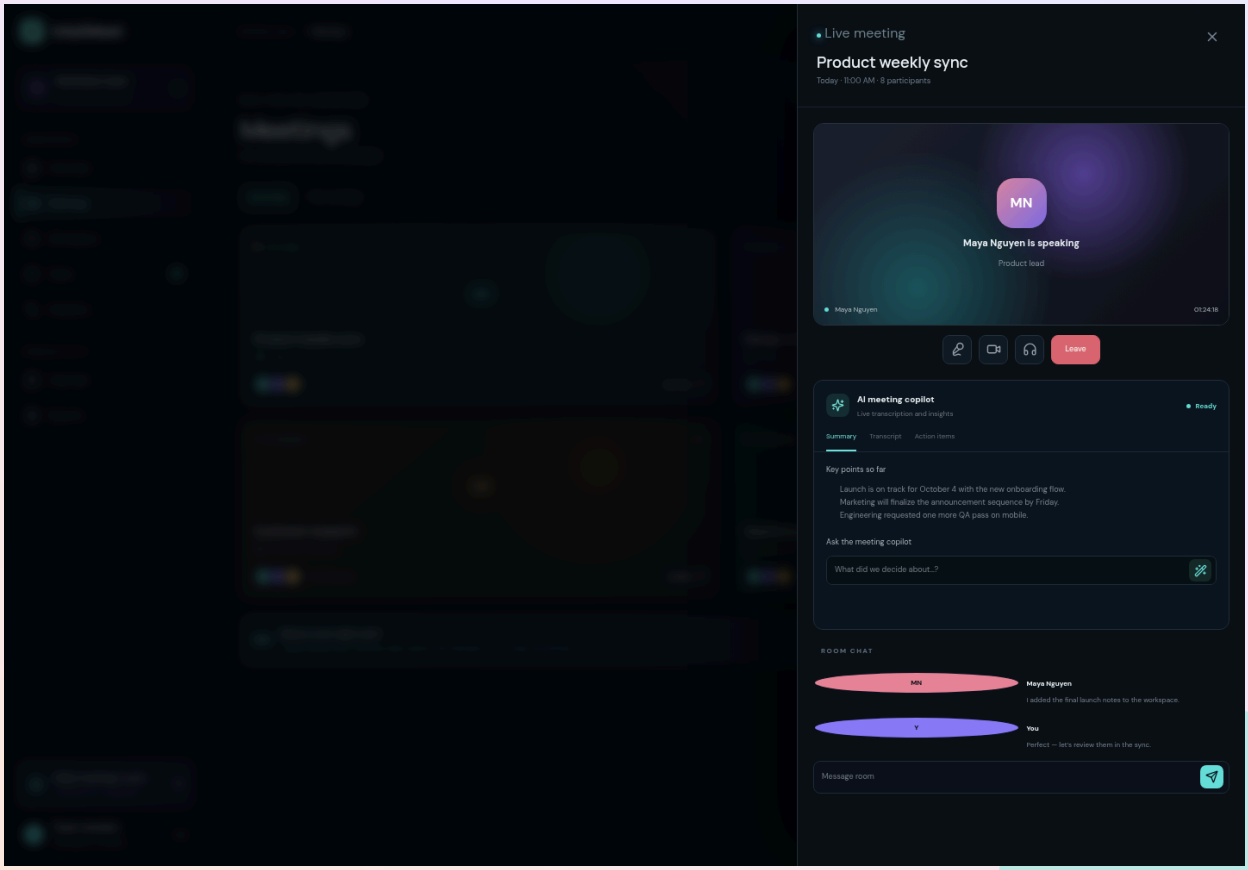


Figure 6: Prototype meeting-room drawer with sample copilot text, transcript context, and room-chat content.

3.4 Team workspace, tasks, and analytics

The Team workspace places open action items next to a team-chat panel to suggest how meeting outcomes could continue after a call. The Tasks view provides an action center for adding and completing follow-up items. The Analytics view visualizes meeting load, response time, and AI-note counts. These areas communicate the intended product experience; the counts and charts are sample values rather than metrics queried from a backend.

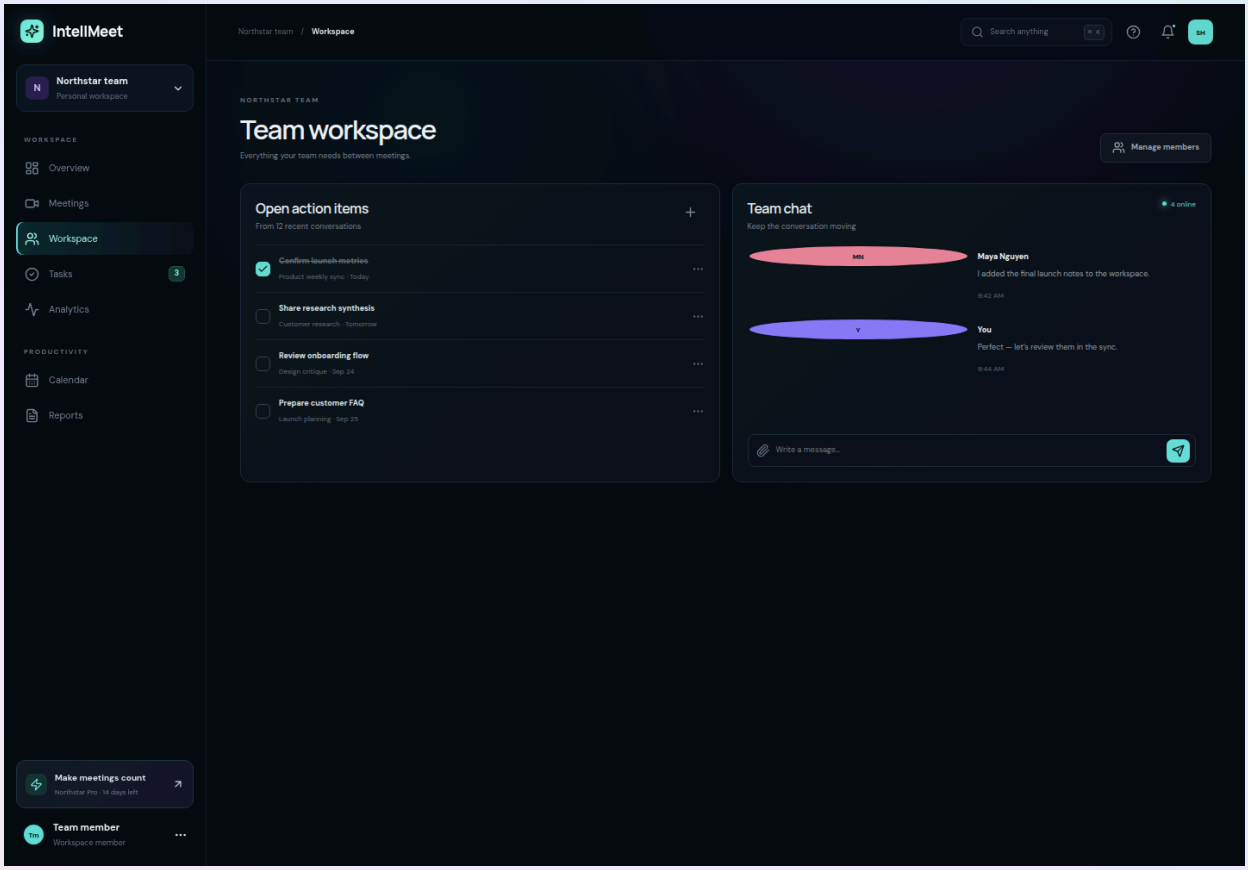


Figure 7: Team workspace prototype with sample action items and team-chat messages.

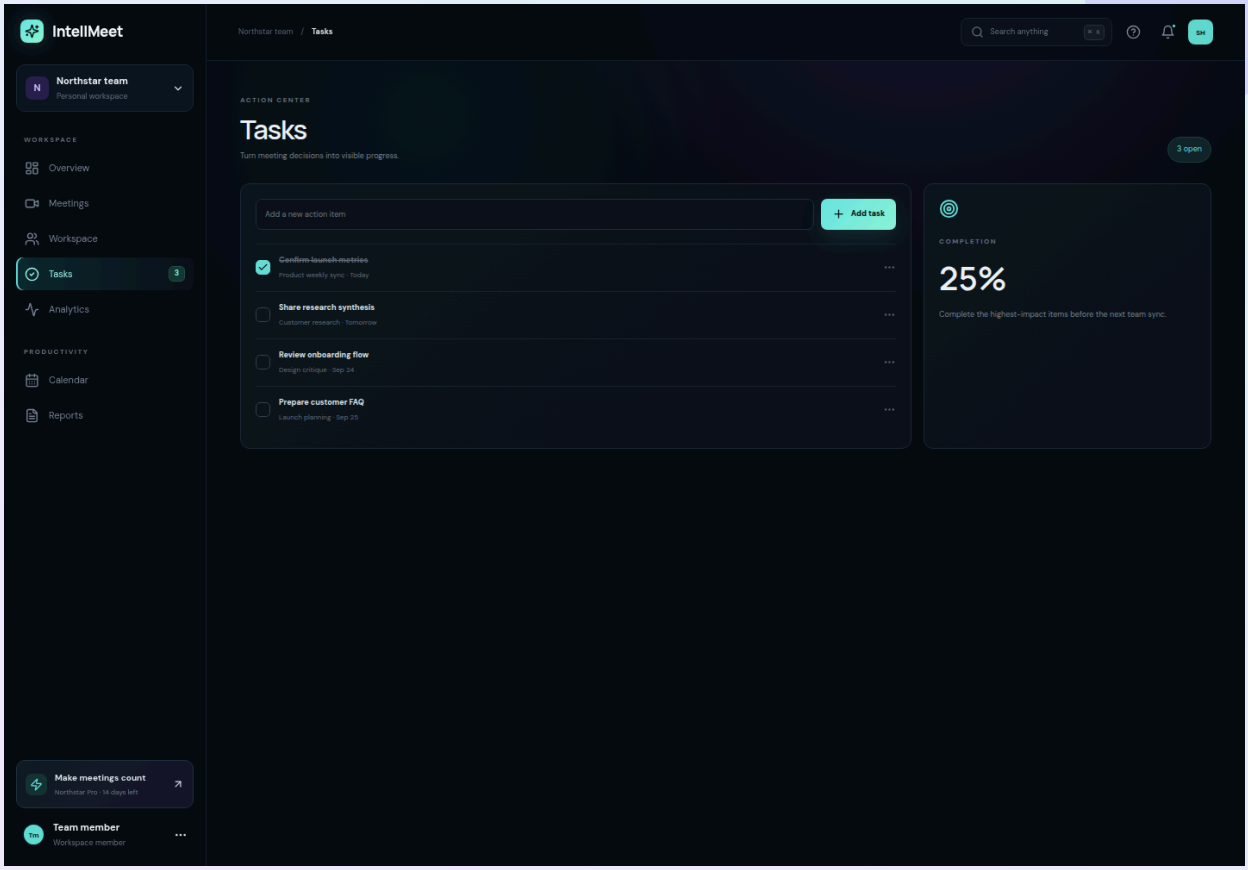


Figure 8: Task center prototype showing completion state and illustrative progress.

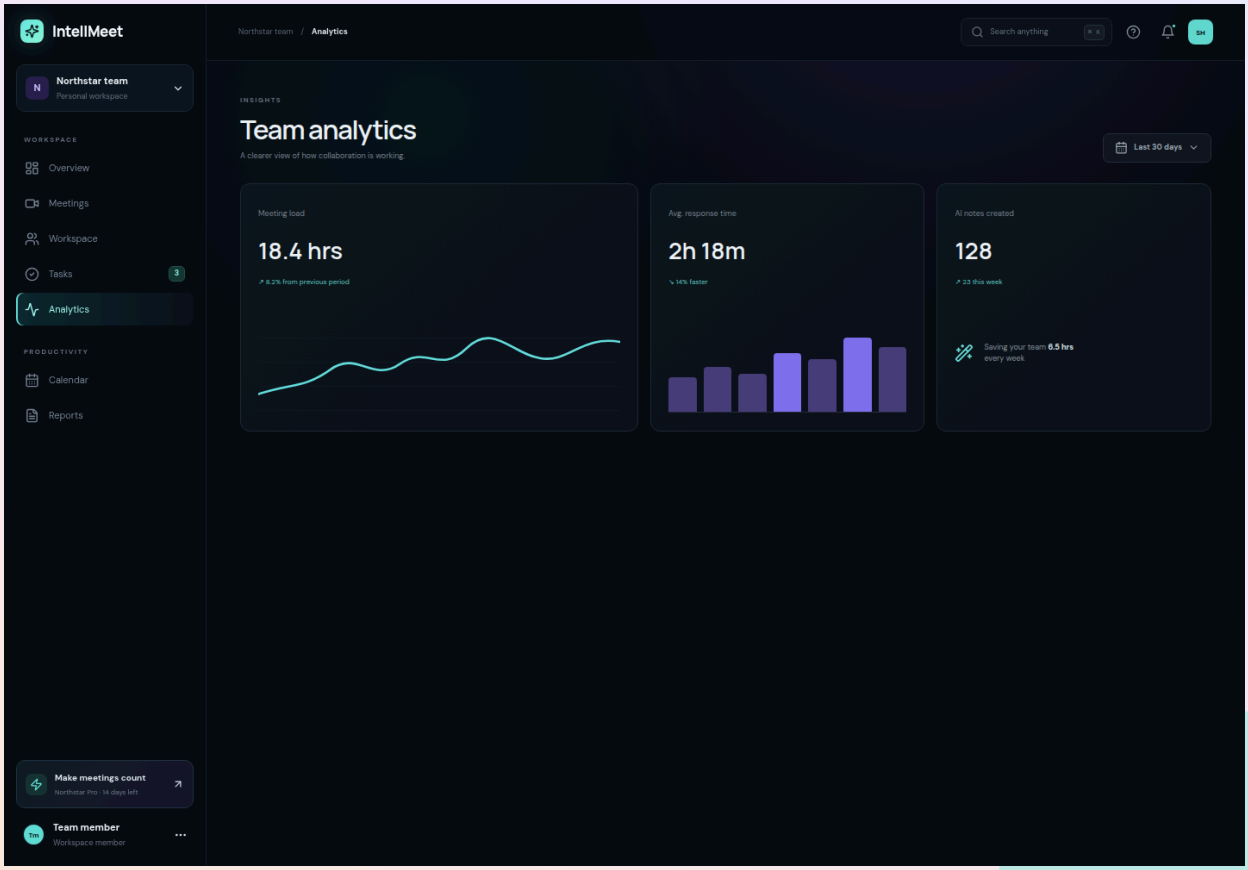


Figure 9: Analytics page with illustrative meeting load, response time, and AI-note values.

4 Feature behavior and persistence

Area	Implemented behavior	Persistence / boundary
Authentication	Create account, sign in, check session, sign out	MySQL/TiDB account tables; requires configuration
Meetings	Browse sample cards, filter/search, create a meeting	Created meetings stored in browser local storage
Tasks	Add and toggle sample action items	Client-side state
Workspace and chat	View sample activity and compose messages	Client-side demonstration only
Analytics and reports	View illustrative metrics and download a text report	Sample values; not server-derived
Meeting room	Open room drawer and interact with demo controls/tabs	No live audio/video, sockets, or saved transcript
Copilot	Search the workspace and run meetings, tasks, analytics, reports, or new rooms from a prompt	Server-side LLM procedure added; existing prototype views remain

5 Technology stack

Layer	Technologies and role
Frontend	React 19 and TypeScript; Wouter for routes; Vite 7 for development and bundling; Tailwind CSS 4 plus project CSS for styling; Lucide icons.
API	Node.js and Express 4 host a tRPC 11 API under `/api/trpc`; shared TypeScript types connect browser calls to procedures.
Data access	Drizzle ORM and mysql2 map user records to MySQL/TiDB. SQL schema is defined in `drizzle/schema.ts`, with migrations under `drizzle/`.
Authentication	bcryptjs hashes passwords at 12 rounds; JOSE signs and verifies HS256 session tokens using the configured `JWT_SECRET`; cookies are HTTP-only.
Verification	TypeScript compiler, Vitest, Vite production build, and Drizzle migration generation.

6 System architecture

The user interface sends typed API calls through the tRPC client to the Express server. The authentication router validates inputs and delegates persistence to Drizzle. During registration, bcrypt hashes the password before an atomic database transaction writes the profile and credential records. Login fetches the normalized email record and compares the submitted password with its bcrypt hash. The server signs a session token and sets the `app_session_id` cookie. Subsequent requests pass through the existing request-context authentication logic, which resolves the user record.

The dashboard's non-authentication data uses local component state or browser storage. The diagram separates that prototype path from the backend-backed account flow so the current delivery boundary is clear.

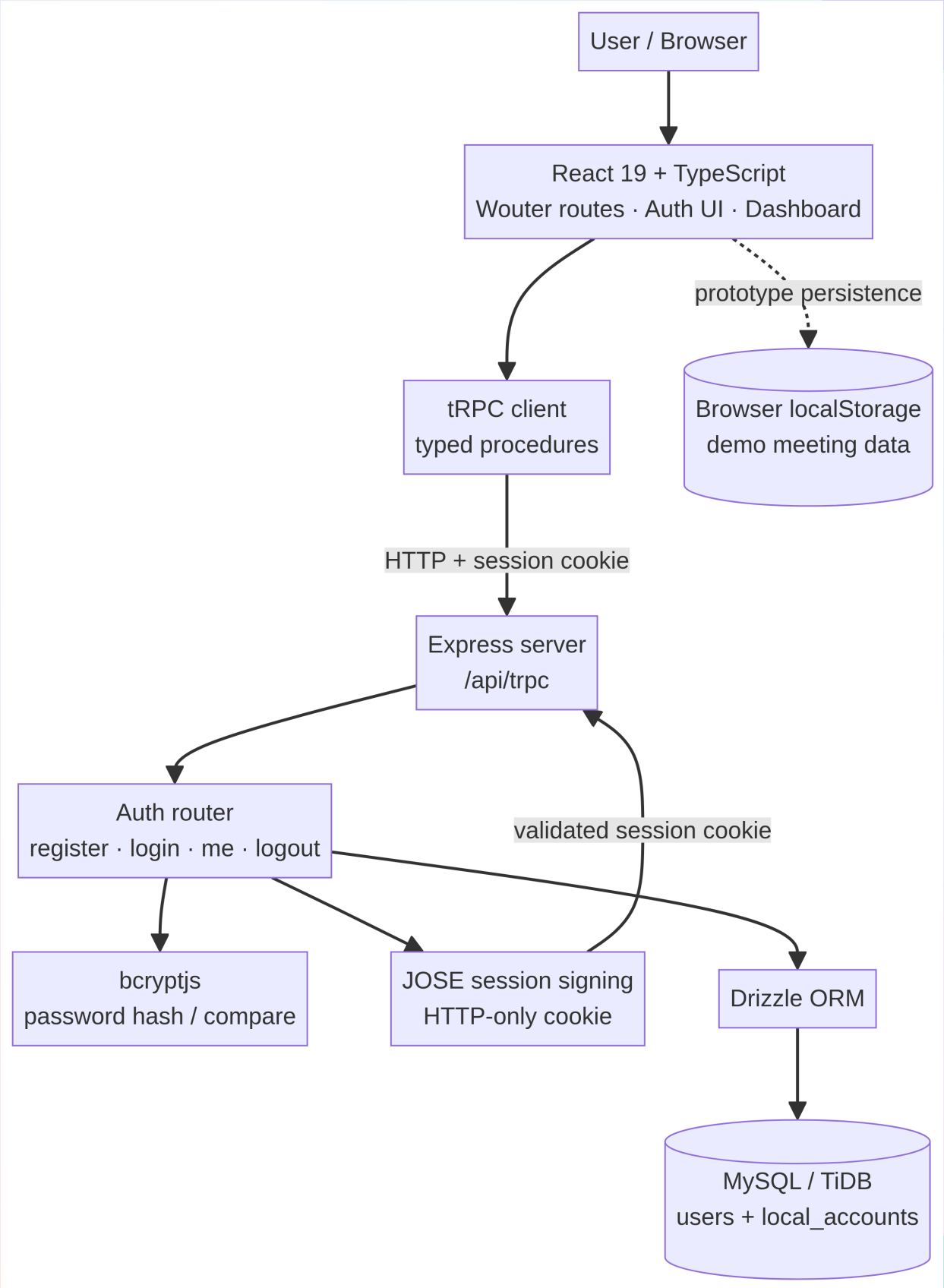


Figure 10: System architecture of the current build. The solid backend path is implemented; persistent meeting collaboration and live media are outside the current scope.

6.1 Registration and sign-in sequence

1. The browser submits form data to `auth.register` or `auth.login` through tRPC.
2. Zod validates inputs; the server normalizes email addresses before lookup or storage.
3. Registration hashes the password with bcrypt and transactionally inserts the user profile and local credential record.
4. Login verifies the hash, updates the sign-in timestamp, and signs a one-year JWT session.
5. The HTTP-only session cookie is attached to requests. The `auth.me` query returns the signed-in profile; logout clears the cookie.

7 Data model and backend procedures

The `users` table holds the profile used by request authentication. The `local_accounts` table stores the normalized email, profile identifier, bcrypt hash, and account creation time. The tables use unique identifiers for `openId` and unique email storage for local accounts. Password hashes are kept separate from the user profile fields returned to the client.

Procedure	Access	Purpose
`auth.register`	Public	Validate and create account; return safe profile; set session cookie.
`auth.login`	Public	Verify email/password; refresh sign-in time; set session cookie.
`auth.me`	Public query with session context	Return the resolved current user, or `null` when signed out.
`auth.logout`	Public	Clear the session cookie.
`system.health`	Public	Return a lightweight backend health result.

8 Security and engineering quality

Passwords are never returned by authentication procedures and are not stored as plaintext. Email is trimmed and lowercased before account lookup. Registration and login reject password inputs exceeding bcrypt's 72-byte limit. Session cookies are HTTP-only. The cookie is Secure for HTTPS requests; local HTTP uses a compatible SameSite setting for development.

The current implementation is a learning prototype, not a complete production identity system. It does not yet include email verification, password reset, multi-factor authentication, account lockout, login rate limiting, automated abuse controls, or short-lived/rotating sessions. Production deployment should use a unique secret of at least 32 bytes, HTTPS, a managed database with backups, rate limiting, monitoring, and a documented recovery path.

9 Validation and deployment status

9.1 Checks completed

Check	Evidence	Result
`pnpm check`	TypeScript compilation of client and server	Passed
`pnpm test`	Three test files; six tests including bcrypt verification and existing meeting/logout tests	Passed
`pnpm build`	Vite production client and bundled Express entry	Passed
Drizzle generation	Two tables detected; generated migration reviewed; no schema changes on repeat run	Passed
Local route/API smoke	`/login` and `/register` returned HTTP 200; `system.health` returned `{ok: true}`	Passed

9.2 Deployment status

The project archive does not include a configured `DATABASE_URL` or production hosting credentials. Therefore the migration has not been applied to a live database, and registration/login have not been verified against a real MySQL/TiDB service in this report run. A temporary local API smoke test confirmed route availability and explicit missing-configuration handling; it did not verify persistent account creation. Public deployment is pending.

To complete deployment, provide a managed MySQL/TiDB database, set `DATABASE_URL` and a unique `JWT_SECRET` in the host's secret manager, apply the Drizzle migration, build with `pnpm build`, and start the Node server with `pnpm start`. Use HTTPS. Then test new registration, duplicate email rejection, correct and incorrect login, session persistence after refresh, sign-out, and signed-out redirection using the deployed URL.

10 Limitations and recommended next steps

The dashboard's meeting list is a sample dataset and meeting creation is browser-local. Tasks, workspace messages, transcripts, analytics, and reports are not persisted per user on the server. The meeting-room surface has no live WebRTC transport or signaling. Copilot text is sample content; the current app does not call an external language model for the displayed transcript. No Socket.IO service is configured.

The new IntellMeet Copilot panel now calls a server-side language-model procedure so users can search the workspace and run actions such as opening meetings, adding tasks, or downloading a report. Sample transcript text in the meeting-room drawer remains illustrative.

A sensible implementation order is to establish a verified deployment and database first, then add user-owned meeting and action tables with protected procedures. Next, connect messages and transcript data to

those records. Only after product scope and infrastructure are clear should the team add real-time events, media transport, and AI transcription/summarization. The project documentation should be updated with test evidence each time these capabilities become real.

11 Conclusion

IntellMeet now has a coherent account-entry flow and a clear interface for moving from meetings to follow-up work. The email registration and login routes, bcrypt password storage, signed cookie sessions, database schema, logout action, and dashboard gate provide a meaningful backend foundation beyond the original frontend-only experience.

The project builds cleanly and its automated checks pass. Its delivery boundary is also explicit: authentication persistence depends on a configured MySQL/TiDB database, and the collaboration features remain prototype interactions. Once a real database is connected and the auth flow is tested on an HTTPS deployment, the project will have stronger evidence for a complete submission. Persisting meeting data and integrating live collaboration would be the next major product milestone.

This restyled edition keeps every original section and screenshot, adds the missing Shubham Singh signed-in dashboard capture, and documents the new Copilot that can search the workspace and run product actions.

12 Alignment with the Zidio web brief

The Zidio IntellMeet brief asks for a production-grade meeting platform covering F-01 to F-07, a MERN-style stack, real-time video, Socket.io, MongoDB, Redis, Kubernetes, and a submission pack of PDF, live HTTPS demo, README, and repository. This build keeps the existing React 19 / Express / tRPC / Drizzle / MySQL stack rather than replacing a working authentication database with MongoDB. The following table maps every required capability to what the website and ZIP actually contain.

ID	Required capability	Implemented in this submission
F-01	Authentication and profiles	Email/password register and login, bcrypt hashing, JWT HTTP-only cookie, dashboard profile, public `/demo` for judges, and a team-invite modal.
F-02	Real-time video meetings	Meeting lobby, room drawer, and a live WebRTC call: getUserMedia camera/mic, Socket.io signaling on `/api/socket`, STUN ICE, screen share, and local recording. Open `/call/<room>` in another tab to join.
F-03	AI meeting intelligence	Meeting-room summary, transcript, and action-item tabs plus a server-side Copilot that can search and run workspace actions.
F-04	Chat and collaboration	Workspace chat and in-meeting chat are interactive in the UI. Socket.io clustering is not configured.
F-05	Post-meeting dashboard	Past meetings, summaries, analytics, and an exportable workspace report.

F-06	Team and project management	Workspace members, invite flow, tasks, and a Kanban board with To do, In review, and Done columns.
F-07	Analytics and insights	Charts, collaboration metrics, and report download. Values remain illustrative prototype data.

Live public demo (no account required): `https://intellmt-b8wygh5b.manus.space/demo`
The brief also lists MongoDB, Redis, Socket.io, WebRTC, Kubernetes, Prometheus, Grafana, and Sentry as the industry target stack. Those production operations layers are not present in this ZIP. The shipped equivalent is a typed full-stack prototype: React 19, TypeScript, Vite, Tailwind, tRPC, Express, Drizzle ORM, MySQL/TiDB, bcrypt, JOSE JWT cookies, Vitest, and a server-side LLM Copilot.

13 References

1. React documentation
2. TypeScript documentation
3. Express documentation
4. tRPC documentation
5. Drizzle ORM MySQL documentation
6. bcrypt.js
7. JOSE documentation